Flask Framework

Building REST APIs with Python

Python + Flask Workshop • Day 3

1. What is Flask?

Flask is a lightweight web framework written in Python. It lets you build web applications and REST APIs quickly with minimal code. Flask is one of the most popular Python frameworks used by startups, companies, and developers worldwide.

Simple Definition

Flask is like a skeleton for your web application. It handles the boring parts (routing requests, serving responses) so you can focus on writing your actual application logic.

Why is Flask so popular?

- Minimal and simple — very easy to learn as a beginner.
- Flexible — you only add what you need, nothing extra.
- Pure Python — if you know Python, you can use Flask.
- Great community and documentation.
- Used in production by companies like Pinterest, LinkedIn, and Netflix.

2. Flask vs Other Frameworks

Feature	Flask	Django	FastAPI
Type	Micro-framework	Full-stack framework	Modern async framework
Learning Curve	Easy	Steep	Medium
Speed	Fast	Medium	Very Fast
Best For	Small/medium APIs	Large web apps	High-performance APIs
Database	Your choice	Built-in Django ORM	Your choice
Used in	Startups, APIs	Instagram, Mozilla	Uber, Microsoft

For This Workshop

We are using Flask because it is the simplest framework to understand how APIs work. Once you understand Flask, learning Django or FastAPI is easy.

3. Installation and Setup

Step 1 — Install Flask

Run this in your terminal / command prompt

```
pip install flask
```

Step 2 — Verify Installation

Terminal

```
python -c "import flask; print(flask.__version__)"
# Should print something like: 3.0.3
```

Step 3 — Install MySQL connector

For connecting Flask to MySQL

```
pip install mysql-connector-python
```

4. Your First Flask Application

Let's build the simplest possible Flask app — a Hello World API.

app.py — Minimal Flask App

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return 'Hello, Students!'
```

```
if __name__ == '__main__':
    app.run(debug=True)
```

Run it

```
Terminal
python app.py
```

Open your browser or Postman and visit: http://localhost:5000
You will see: Hello, Students!

Line-by-Line Explanation

Code	What it does
`from flask import Flask`	Import the Flask class from the flask library.
`app = Flask(__name__)`	Create a Flask application instance. __name__ tells Flask the name of the current file.
`@app.route('/')`	A decorator that tells Flask: when someone visits '/', call the function below it.
`def hello():`	A regular Python function. This runs when the route is triggered.
`return 'Hello, Students!'`	Send this text back as the response to the client.
`app.run(debug=True)`	Start the development server. debug=True shows errors in the browser and auto-restarts on code changes.

5. Routing — How URLs Map to Functions

Routing is the process of mapping a URL to a Python function. Each route in Flask is defined using the @app.route() decorator.

```
app.py — Routing Examples
from flask import Flask

app = Flask(__name__)

# Route 1 - Home page
@app.route('/')
def home():
```

```
    return 'Welcome to the Student API!'

# Route 2 - Fixed path
@app.route('/students')
def get_students():
    return 'List of all students'

# Route 3 - Dynamic path (URL parameter)
@app.route('/students/<int:student_id>')
def get_student(student_id):
    return f'Fetching student with ID: {student_id}'

if __name__ == '__main__':
    app.run(debug=True)
```

URL Parameter — <int:student_id>

The <int:student_id> part in the route is a dynamic segment. When someone visits /students/5, Flask automatically captures 5 and passes it to the function as the variable student_id. The 'int:' part means Flask will only accept integer values there.

6. Handling HTTP Methods in Flask

By default, Flask routes only accept GET requests. To handle POST, PUT, DELETE etc., you must specify them in the methods parameter of @app.route().

app.py — All HTTP Methods

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# GET — Read data
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify({'message': 'Returning all students'})

# POST — Create new data
@app.route('/students', methods=['POST'])
def add_student():
    data = request.get_json()   # Read JSON from request body
    name = data['name']
    return jsonify({'message': f'Added student: {name}'}), 201

# PUT — Update existing data
@app.route('/students/<int:id>', methods=['PUT'])
def update_student(id):
    data = request.get_json()
    return jsonify({'message': f'Updated student {id}'}), 200
```

```
# DELETE — Remove data
@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):
    return jsonify({'message': f'Deleted student {id}'}), 200

if __name__ == '__main__':
    app.run(debug=True)
```

7. The Three Most Important Flask Objects

A) jsonify() — Send JSON Responses

jsonify() converts a Python dictionary into a proper JSON HTTP response. Always use jsonify() instead of returning a plain string for API responses.

```
from flask import jsonify

# Without jsonify (BAD for APIs)
return {'name': 'Rahul'}          # Returns a plain dict string

# With jsonify (CORRECT)
return jsonify({'name': 'Rahul'}) # Returns proper JSON response with
correct headers
```

B) request — Read Incoming Data

The request object holds all data sent by the client. Use it to read JSON body, URL parameters, and headers.

```
from flask import request

# Read JSON body (from POST/PUT requests)
data = request.get_json()
name = data['name']

# Read URL query parameter (?search=rahul)
search = request.args.get('search')

# Read a specific header
content_type = request.headers.get('Content-Type')
```

C) app.route() decorator — Define Routes

The @app.route() decorator registers a URL path and the allowed HTTP methods for that path.

```
# methods tells Flask which HTTP verbs this route accepts
@app.route('/students', methods=['GET', 'POST'])
def students():
    if request.method == 'GET':
        return jsonify({'action': 'fetching students'})
    elif request.method == 'POST':
        return jsonify({'action': 'adding student'}), 201
```

8. Returning Status Codes in Flask

Flask lets you return a status code as the second value in the return statement. If you do not specify one, Flask defaults to 200 OK.

app.py — Status Code Examples

```
# 200 OK (default - no need to write it but good practice)
return jsonify({'message': 'Success'}), 200

# 201 Created (use after adding a new record)
return jsonify({'message': 'Student added'}), 201

# 400 Bad Request (invalid data from client)
return jsonify({'error': 'Name is required'}), 400

# 404 Not Found (record does not exist)
return jsonify({'error': 'Student not found'}), 404

# 500 Internal Server Error (something crashed in server)
return jsonify({'error': 'Database error'}), 500
```

9. Complete CRUD Example — Student API

Here is a complete, working Flask API with all 4 CRUD operations using an in-memory list (no database needed for this demo). This is the pattern you will use with MySQL.

app.py — Full CRUD (in-memory)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# In-memory storage (we use MySQL in the real project)
```

```
students = []
next_id  = 1

# ── CREATE ──────────────────────────────────────
@app.route('/students', methods=['POST'])
def add_student():
    global next_id
    data    = request.get_json()
    student = {'id': next_id, 'name': data['name'], 'roll':
data['roll']}
    students.append(student)
    next_id += 1
    return jsonify({'message': 'Added', 'student': student}), 201

# ── READ ────────────────────────────────────────
@app.route('/students/<int:sid>', methods=['GET'])
def get_student(sid):
    student = next((s for s in students if s['id'] == sid), None)
    if not student:
        return jsonify({'error': 'Not found'}), 404
    return jsonify(student), 200

# ── UPDATE ──────────────────────────────────────
@app.route('/students/<int:sid>', methods=['PUT'])
def update_student(sid):
    data    = request.get_json()
    student = next((s for s in students if s['id'] == sid), None)
    if not student:
        return jsonify({'error': 'Not found'}), 404
    student['name'] = data.get('name', student['name'])
    return jsonify({'message': 'Updated', 'student': student}), 200

# ── DELETE ──────────────────────────────────────
@app.route('/students/<int:sid>', methods=['DELETE'])
def delete_student(sid):
    global students
    found    = any(s['id'] == sid for s in students)
    if not found:
        return jsonify({'error': 'Not found'}), 404
    students = [s for s in students if s['id'] != sid]
    return jsonify({'message': 'Deleted'}), 200

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

10. Recommended Project Structure

For the workshop mini project (with MySQL), organize your files like this:

```
student_result_api/
  ├── app.py          ←  All Flask routes (API logic)
  ├── db.py           ←  MySQL connection helper
  └── schema.sql      ←  Database structure + sample data
```

Why separate db.py from app.py?

- Single Responsibility — each file has one job.
- Reusability — any route in app.py can import and use get_connection() from db.py.
- Easy to maintain — if DB credentials change, update only db.py.

11. Running, Stopping, and Debugging

Start the Server

Terminal
```
python app.py
```

You will see output like:

```
 * Running on http://127.0.0.1:5000
 * Debug mode: on
 * Restarting with stat
```

Stop the Server

```
Press Ctrl + C in the terminal
```

debug=True — What it does

- Shows a detailed error page in the browser if something crashes.
- Automatically restarts the server when you save changes to your code.
- NEVER use debug=True in production — only in development.

Common Errors and Fixes

Error	Fix
`Address already in use`	Another process is using port 5000. Change port: app.run(port=5001)

`ModuleNotFoundError: flask`	Run: pip install flask
`404 Not Found`	The URL you visited does not match any route. Check your @app.route() path.
`400 Bad Request`	You sent invalid or missing data. Check the request body in Postman.
`500 Internal Server Error`	Your Python code crashed. Read the error message in the terminal.
`get_json() returns None`	You forgot to set Content-Type: application/json in Postman headers.

12. Flask Quick Reference Cheat Sheet

What you want to do	Flask Code
Create Flask app	`app = Flask(__name__)`
Define a GET route	`@app.route('/path', methods=['GET'])`
Define a POST route	`@app.route('/path', methods=['POST'])`
URL with variable	`@app.route('/students/<int:id>')`
Read JSON request body	`data = request.get_json()`
Read URL query param (?key=v)	`val = request.args.get('key')`
Send JSON response	`return jsonify({'key': 'value'})`
Send response with status code	`return jsonify({...}), 201`
Run the app	`app.run(debug=True, port=5000)`
Import all key tools	`from flask import Flask, request, jsonify`

You now know everything you need to build REST APIs with Flask!

Start small, break things, learn fast.